

Git Commands Visual Guide

The everyday commands, explained with pictures

A reference for complete beginners

Plain-English glossary of Git words

Diagrams: workflow, branches, push, merge, pull

Every command with a one-line explanation

A typical daily workflow you can copy

No experience needed - follow along one step at a time.

Words you will see (plain-English glossary)

Git has its own vocabulary. Here is every word used in this guide, in normal human language:

Repository (repo)

your project folder plus its whole saved history. Think of it as the project with a memory.

Commit

a saved snapshot of your project at one moment, with a short message describing what changed. Like a save point in a video game.

Staging area

a waiting room where you gather changes before saving them as a commit. git add puts changes here.

Remote

a copy of your repository that lives somewhere else - usually on GitHub. Your own copy is called local.

Branch

a named side-track of commits. It lets you work on something new without touching the main line of the project.

main

the default name of the primary branch - the official line of the project.

Merge

combining the work from one branch back into another, so both sets of changes end up together.

Push

uploading your local commits to the remote (GitHub).

Pull

downloading new commits from the remote and adding them to your local copy (and your files).

Fetch

downloading new commits from the remote WITHOUT changing your files yet - a safe look before you pull.

Clone

downloading a complete copy of a remote repository, history included, to your computer.

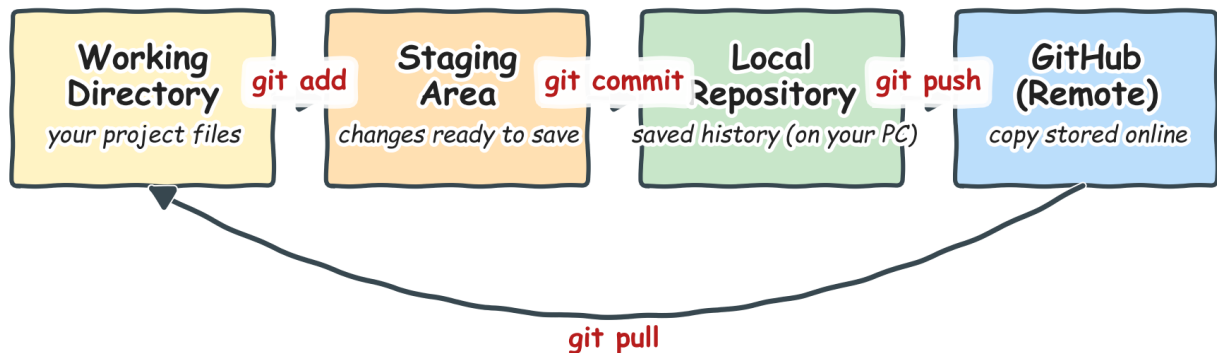
origin

the default nickname Git gives to the remote your repository is connected to - usually your GitHub URL.

The big picture: where your code travels

Every day with Git, your changes move through four places. The commands on the arrows are the ones that move your work from one place to the next:

The basic Git workflow: where your code travels

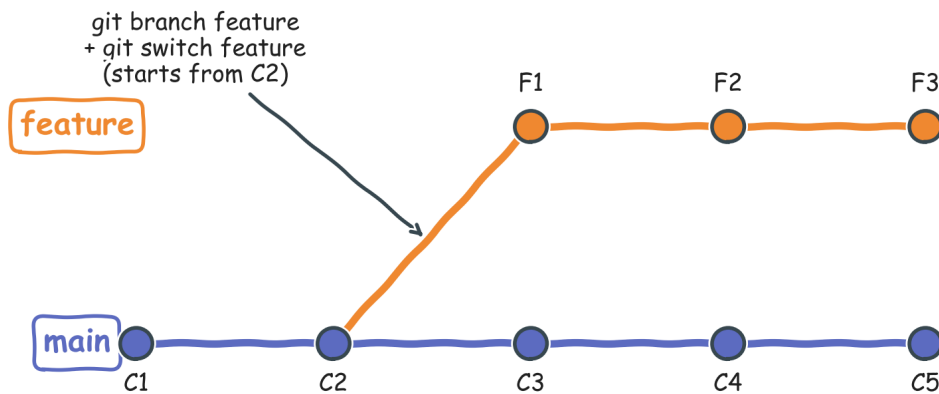


- You edit files in the working directory (just your normal folder).
- `git add` moves chosen changes into the staging area (the waiting room).
- `git commit` saves what is staged into your local repository (a permanent snapshot in the history).
- `git push` copies your commits up to GitHub so they are backed up and shareable.
- `git pull` brings other people's commits down from GitHub into your copy.

How branches work

A branch is a safe side-track. You create one, do your new work there, and the main line stays untouched until you decide otherwise. In the picture, each circle is a commit (a saved snapshot):

Branches: a safe side-track for new work

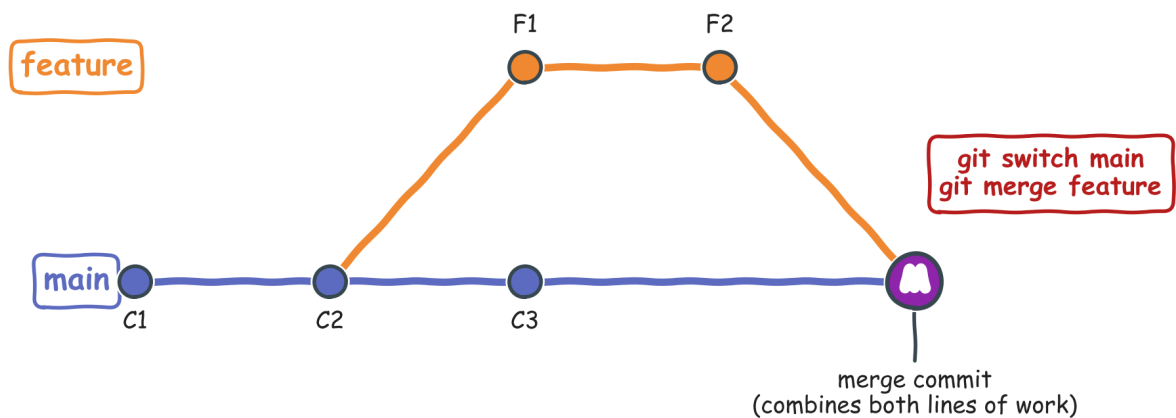


- C1 to C5 are commits on the main branch - the official line.
- The feature branch starts from C2 and gets its own commits F1, F2, F3.
- While you work on feature, main can keep moving (C3, C4, C5) - nothing on your branch breaks it, and nothing on main breaks your branch.

How merging works

When the work on your branch is finished, you merge it: Git combines the branch back into main and creates a merge commit that ties the two lines of history together:

Merging: bringing the branch back into main



- Switch to the receiving branch first: `git switch main`.
- Then combine: `git merge feature` creates the merge commit M.
- Afterwards the branch can be deleted: `git branch -d feature`. The commits stay in the history.

What happens when you push

Committing only saves to YOUR computer. Pushing copies those commits up to GitHub, so they are backed up online and visible to your teammates:

What git push does

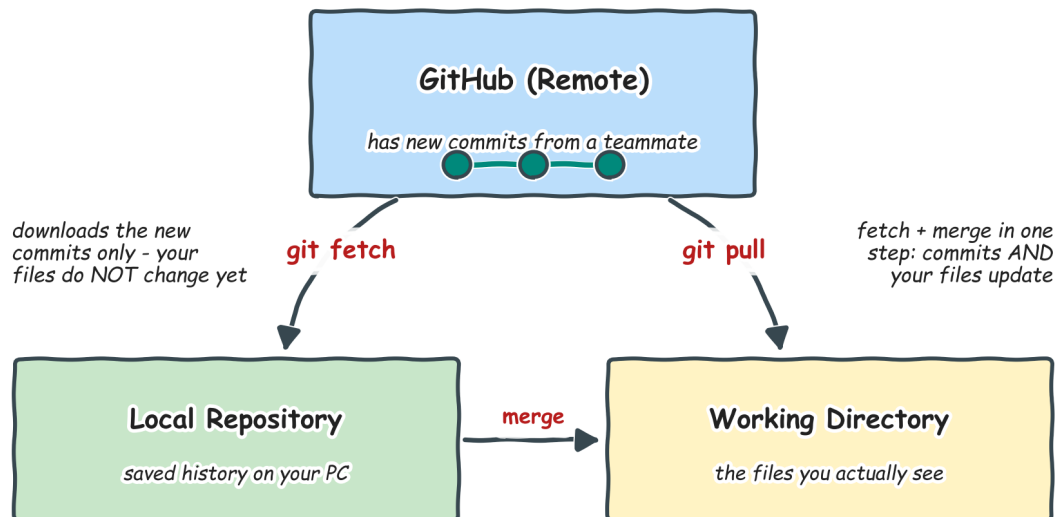


- Nothing reaches GitHub until you run git push.
- If GitHub already has newer commits from someone else, push is refused - run git pull first, then push again.

Pull and fetch: getting changes FROM GitHub

These two commands move commits in the opposite direction - from GitHub down to your computer. The difference is how much they change:

git fetch vs git pull: getting changes FROM GitHub



- git fetch only downloads the new commits into your local repository. Your files stay exactly as they were - perfect for checking what changed before accepting it.
- git pull is git fetch followed by git merge in one step: it downloads AND updates your files. This is the one you will use most mornings.

Command reference: setup & configuration

git --version

Shows which version of Git is installed - the quickest way to check Git works.

git config --global user.name "Your Name"

Sets the name that gets stamped on every commit you make (one-time setup per computer).

git config --global user.email "you@example.com"

Sets the email stamped on your commits - use the same one as your GitHub account.

git config --list

Shows all your current Git settings, so you can check what you configured.

git help <command>

Opens the manual page for a command (e.g. git help commit) when you want the full details.

Command reference: starting a project

git init

Turns the current folder into a Git repository - the very first step for a brand-new project on your computer.

git clone <url>

Downloads a complete copy of a GitHub repository (files plus whole history) into a new folder.

git status

Shows what is going on right now: which files changed, what is staged, and which branch you are on. Use it constantly.

Command reference: staging & committing

git add <file>

Stages one file - puts its changes in the waiting room, ready for the next commit.

git add .

Stages every changed file in the project at once. Handy, but run git status first so you know what you are including.

git commit -m "describe the change"

Saves everything staged as a snapshot with your message. Write the message as if explaining the change to a colleague.

git commit -am "message"

Stages modified tracked files AND commits in one step (new files still need git add first).

git log

Shows the list of past commits, newest first (press q to quit the viewer).

git log --oneline

Same history, but one compact line per commit - great for a quick overview.

git diff

Shows exactly which lines changed in files that are not staged yet.

git diff --staged

Shows which changes are currently staged and about to be committed.

Command reference: undoing changes

Undoing in Git depends on how far the change has travelled. These are ordered from most harmless to most powerful:

git restore <file>

Throws away unstaged edits in a file, putting it back the way the last commit left it. (The edits are gone for good.)

git restore --staged <file>

Takes a file OUT of the staging area without touching its contents - for when you added something by accident.

git commit --amend -m "new message"

Rewrites your MOST RECENT commit - fix its message or add files you forgot. Only do this before pushing.

git reset --soft HEAD~1

Un-does the last commit but keeps all its changes staged, ready to commit again differently. Only before pushing.

git revert HEAD

Creates a NEW commit that cancels out the last one - the safe way to undo something that was already pushed.

Note: Golden rule: once a commit is pushed, prefer git revert. Rewriting shared history (reset, amend) confuses everyone who already pulled it.

Command reference: branching & merging

git branch

Lists all local branches - the one with a star is where you are now.

git branch <name>

Creates a new branch pointing at your current commit, but stays where you are.

git switch <name>

Jumps to another branch; your files change to match it.

git switch -c <name>

Creates a new branch AND jumps to it in one step - the usual way to start new work.

git merge <name>

Combines the named branch into the branch you are on now, creating a merge commit.

git branch -d <name>

Deletes a branch that has been merged (safe - Git refuses if work would be lost).

git merge --abort

Bails out of a merge that hit conflicts, putting everything back the way it was before you started.

Note: Older tutorials use git checkout <name> instead of git switch. Both work; switch is the modern, clearer command.

Command reference: remotes (GitHub)

git remote -v

Shows the GitHub URLs your repository is linked to.

git remote add origin <url>

Links your local repository to a GitHub repository and nicknames it origin (done once, right after git init).

git push

Uploads your committed snapshots to GitHub.

git push -u origin main

First push of a branch: uploads it AND remembers the link, so later a plain git push is enough.

git pull

Downloads new commits from GitHub and merges them into your current branch (fetch + merge in one step).

git fetch

Downloads new commits from GitHub WITHOUT changing your files - a safe preview of what teammates did.

git clone <url>

Downloads a whole GitHub repository to your computer, already linked to its remote.

Command reference: stashing

Stashing shelves your unfinished work so you can quickly switch tasks, then brings it back later.

git stash

Shelves all uncommitted changes and gives you a clean working directory - like sweeping the desk into a drawer.

git stash list

Shows everything currently sitting in the drawer.

git stash pop

Brings the most recent stash back into your files and removes it from the drawer.

git stash apply

Like pop, but keeps a copy in the drawer in case you need it again.

git stash drop

Throws away the most recent stash without applying it.

Command reference: tags

A tag is a sticky note attached to one commit - most often used to mark releases like v1.0.

git tag

Lists all tags in the repository.

git tag -a v1.0 -m "first release"

Creates a named tag with a message on the current commit.

git push origin v1.0

Uploads one tag to GitHub (tags do not travel with a normal push).

git push origin --tags

Uploads all your tags to GitHub at once.

A typical daily workflow

This is the rhythm you will repeat every day. Commands on the left, what is happening on the right:

1. `git pull`

Start the day by downloading any new commits from GitHub.

2. `git switch -c my-feature`

Create and jump to a fresh branch for today's task.

3. `...edit your files...`

Do the actual work - code, write, fix.

4. `git status`

Check which files changed; make sure nothing extra sneaks in.

5. `git add .`

Stage all the changes you want to save.

6. `git commit -m "Add login form validation"`

Save the snapshot with a clear message.

7. `git push -u origin my-feature`

Upload the branch to GitHub.

Then, on the GitHub website: open the repository, click the button to create a Pull Request (a request to merge your branch into main), ask someone to review it, and click Merge when it is approved. Back in your terminal:

```
git switch main
git pull
git branch -d my-feature
```

That updates your local main with the merged work and tidies away the finished branch. Tomorrow, the loop starts again.

Note: A pull request is not a git command - it is a GitHub website feature for reviewing and merging branches with your team.